문제 1

설정 바깥 직사각형 (0,0)-(10,8), 구멍 두 개 (2,2)-(4,6)과 (6,2)-(8,6). 원래 정점 12개.

Steiner 정점 2개: (5,0)과 (5,8). 바깥 아래쪽 변과 위쪽 변 위, 두 구멍 사이 막대(x=4~6)

Steiner 없이 12개 정점만 쓰면 막대 윗변 (4,6)-(6,6)이 삼각분할에 들어갈 때, 이 변 위쪽 삼각형의 세 번째 꼭짓점은 위쪽 바깥 모서리 (0,8)이나 (10,8)밖에 될 수 없음(구멍의 다른 위쪽 꼭짓점은 같은 직선 y=6 위라서 삼각형이 안 만들어짐)

그러면 삼각형 (0,8),(4,6),(6,6)이 생기고 (0,8)에서의 각은 atan(1/2) − atan(1/3) = 26.57° − 18.43° = 8.13°로 가늘고 긴 삼각형이 됨. 아래쪽에서도 막대 아래 변 (4,2)-(6,2)에 대해 같은 모양의 삼각형이 생김

막대 바로 위·아래의 바깥 변에 (5,8), (5,0)을 찍으면 삼각형 (5,8),(4,6),(6,6)의 각이 53.13°, 63.43°, 63.43°가 되고, 전체에서 가장 작은 각이 18.43°.

점이 구멍 사이 막대의 가운데(x=5)에 있어야 좌우 대칭으로 각이 고르게 나온다

삼각형 수 검산: 삼각형 수 = B + 2h − 2 + 2I (경계 정점 B, 구멍 수 h, 내부 정점 I)

- 경계 정점 12 + 2 = 14, 구멍 2 개, 내부 정점 0 → 14 + 4 − 2 = 16 개 (코드의 삼각형 16 개와 일치)

삼각형 목록 (정점 번호, 모두 반시계)

{3,0,4} {3,4,7} {3,7,13} {13,7,6} {13,6,11} {13,11,10} {13,10,2} {2,10,1}{1,10,9} {1,9,12} {12,9,8} {12,8,5} {12,5,4} {12,4,0} {6,5,8} {6,8,11}

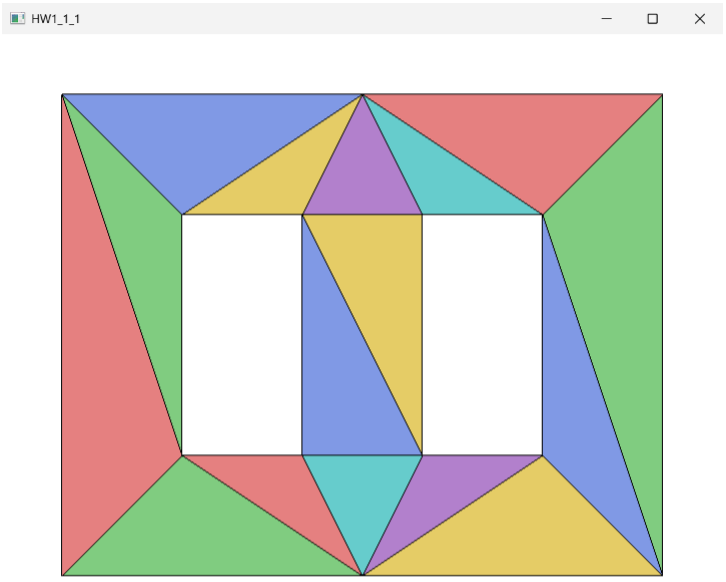
번호: 0~3 바깥 (0,0),(10,0),(10,8),(0,8) / 4~7 왼쪽 구멍 (2,2),(4,2),(4,6),(2,6) / 8~11 오른쪽 구멍 (6,2),(8,2),(8,6),(6,6) / 12 = (5,0), 13 = (5,8)

16개 모두 CCW → OpenGL 기본값(CCW = front) 기준 전부 front. 코드에서 `glPolygonMode(GL_FRONT, GL_FILL)`, `glPolygonMode(GL_BACK, GL_LINE)`으로 설정했더니 16개가 전부 색으로 채워졌다. back인 삼각형이 있었다면 그 삼각형만 선으로 보임

삼각형이 보이도록 6가지 색을 돌아가며 칠하기

삼각형끼리 교차하지 않고 구멍 안에 삼각형X

코드 설명: 정점 좌표를 삼각형 16개(삼각형 하나에 정점 3개, 모두 반시계 순서)의 배열로 만들어 GL_TRIANGLES로 그림. 도형이 (0,0)-(10,8)이라서 glOrtho(-1,11,-1,9,-1,1)로 도형 주변에 여백을 두었고, 삼각형마다 색을 바꿔 칠한 뒤 경계선을 한 번 더 그려 삼각형 경계가 보이게 함.


HW1_1_1

문제 2

설정: E = (3, 2, 4), A = (0, 2, 0), Up = (0, 1, 0)

los = A − E = (−3, 0, −4)

|E − A| = √(9 + 0 + 16) = 5

z_cam = (E − A)/|E − A| = (3, 0, 4)/5 = (0.6, 0, 0.8)

x_cam = Up × z_cam = (0,1,0) × (0.6,0,0.8)

x: 1·0.8 − 0·0 = 0.8

y: 0·0.6 − 0·0.8 = 0

z: 0·0 − 1·0.6 = −0.6

= (0.8, 0, −0.6) (이미 크기가 1이라 정규화해도 그대로)

y_cam = z_cam × x_cam = (0.6,0,0.8) × (0.8,0,−0.6)

x: 0·(−0.6) − 0.8·0 = 0

y: 0.8·0.8 − 0.6·(−0.6) = 0.64 + 0.36 = 1

z: 0.6·0 − 0·0.8 = 0

= (0, 1, 0)

V=R X T

T = translate(−3, −2, −4)

R = 0.8 0 −0.6 0 ← x_cam
0 1 0 0 ← y_cam
0.6 0 0.8 0 ← z_cam
0 0 0 1

V 마지막 열 = −(x_cam·E, y_cam·E, z_cam·E)

x_cam·E = 0.8·3 + 0·2 + (−0.6)·4 = 2.4 − 2.4 = 0

y_cam·E = 0·3 + 1·2 + 0·4 = 2

z_cam·E = 0.6·3 + 0·2 + 0.8·4 = 1.8 + 3.2 = 5

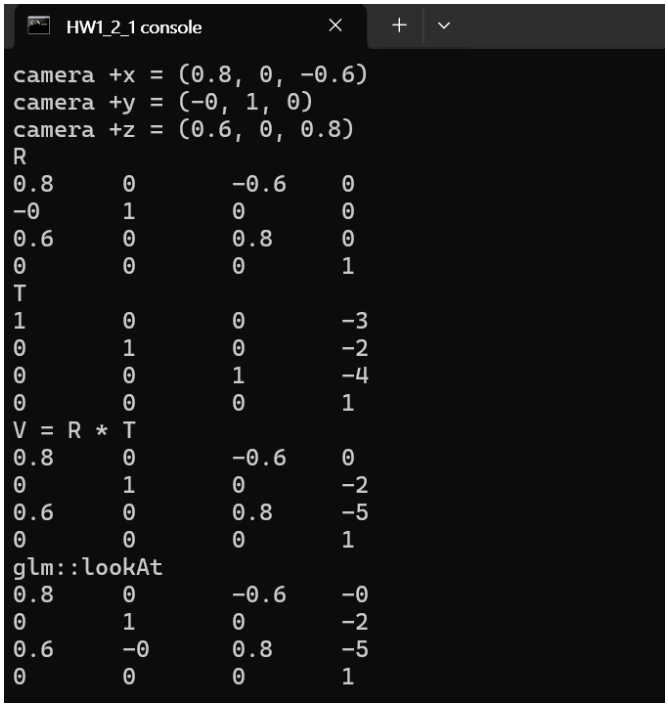
V = 0.8 0 −0.6 0
0 1 0 −2
0.6 0 0.8 −5
0 0 0 1

검산: V × E = (0, 0, 0), V × A = (0, 0, −5). 5 = |E − A| (A가 카메라 앞 −z 방향 거리 5에 있음)

코드와 비교: 직접 만든 V = R*T와 `glm::lookAt(E, A, Up)` 결과가 같음(최대 차이 0). `glm::mat4`는 열 우선이라 출력할 때 `m[열][행]`으로 읽어야 손 계산과 같은 모양이 나온다

출력에 보이는 `-0`은 부동소수점의 음수 0이라 값은 0과 같다

코드 설명: E, A, Up으로 카메라 축 세 개를 외적으로 구하고(z_cam → x_cam → y_cam 순서), 이 축들로 R 행렬을, E로 T 행렬을 직접 만들어 V = R*T를 계산함. 같은 입력으로 glm::lookAt을 호출해서 두 행렬을 나란히 출력해 비교함.


HW1_2_1 console
camera +x = (0.8, 0, -0.6)
camera +y = (-0, 1, 0)
camera +z = (0.6, 0, 0.8)
R
0.8 0 -0.6 0
-0 1 0 0
0.6 0 0.8 0
0 0 0 1
T
1 0 0 -3
0 1 0 -2
0 0 1 -4
0 0 0 1
V = R * T
0.8 0 -0.6 0
0 1 0 -2
0.6 0 0.8 -5
0 0 0 1
glm::lookAt
0.8 0 -0.6 -0
0 1 0 -2
0.6 -0 0.8 -5
0 0 0 1

문제 3

설정: 과제 그림의 U자 도형. 꼭짓점 12개로 이루어진 구멍 없는 다각형

- 왼쪽 위 날개 (0,3)-(2,4), 왼쪽 기둥 (1,0)-(2,4), 아래 막대 (1,0)-(4,1), 오른쪽 기둥 (3,0)-(4,4), 오른쪽 위 날개 (3,3)-(5,4)
- 넓이 = 2 + 3 + 1 + 3 + 2 = 11

최소 정점 수: 구멍 없는 n각형은 삼각형이 n − 2 = 12 − 2 = 10개. strip은 정점 k개로 삼각형 k − 2개를 만드니까 k = 12. 코너 12개를 각각 한 번씩만 쓰므로 최소 정점 수는 12개(반복 정점 없음).

정점 순서 (v0 ~ v11):

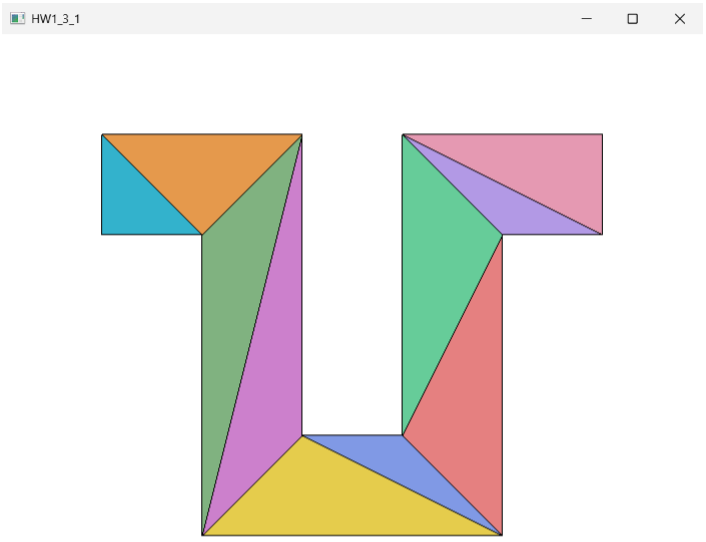
(0,3) (0,4) (1,3) (2,4) (1,0) (2,1) (4,0) (3,1) (4,3) (3,4) (5,3) (5,4)

strip은 짝수 번째 삼각형(2, 4, ...)에서 앞 두 점을 바꿔 쓰므로(예: 2번째는 v2, v1, v3 순서), 표에는 GL이 실제로 쓰는 순서로 외적 z 값을 계산함

삼각형	정점	GL이 쓰는 순서	외적 z	방향
1	v0 v1 v2	(0,3) (0,4) (1,3)	−1	CW
2	v1 v2 v3	(1,3) (0,4) (2,4)	−2	CW
3	v2 v3 v4	(1,3) (2,4) (1,0)	−3	CW
4	v3 v4 v5	(1,0) (2,4) (2,1)	−3	CW
5	v4 v5 v6	(1,0) (2,1) (4,0)	−3	CW
6	v5 v6 v7	(4,0) (2,1) (3,1)	−1	CW
7	v6 v7 v8	(4,0) (3,1) (4,3)	−3	CW
8	v7 v8 v9	(4,3) (3,1) (3,4)	−3	CW
9	v8 v9 v10	(4,3) (3,4) (5,3)	−1	CW
10	v9 v10 v11	(5,3) (3,4) (5,4)	−2	CW

→ 10개 전부 CW로 같은 방향임을 확인했다. 따라서 모든 삼각형의 orientation을 같게 만들 수 있다.

코드에서 `glPolygonMode(GL_BACK, GL_FILL)`, `glPolygonMode(GL_FRONT, GL_LINE)`으로 설정했더니 10개가 전부 색으로 채워짐을 확인. 즉 10개 모두 back 삼각형이라는 뜻이다.



코드 설명: 정점 12개를 strip 순서대로 배열에 넣고 GL_TRIANGLE_STRIP으로 한 번에 그림. 삼각형마다 색이 다르게 보이도록 glShadeModel(GL_FLAT)을 쓰고(삼각형 색은 마지막 정점의 색), 정점마다 색을 지정함. 도형이 (0,0)-(5,4)라서 glOrtho(-1,6,-1,5,-1,1)로 잡음.

문제 4

(a) gluLookAt(0,0,0, −15,0,15, 0,1,0)

eye = (0,0,0), center = (−15,0,15), up = (0,1,0)

los = center − eye = (−15, 0, 15)

z_cam = (eye − center)/|...| = (15,0,−15)/(15√2) = (1/√2, 0, −1/√2)

x_cam = up × z_cam = (0,1,0) × (a,0,−a) (a = 1/√2)

x: 1·(−a) − 0·0 = −a

y: 0·a − 0·(−a) = 0

z: 0·0 − 1·a = −a

= (−a, 0, −a) (크기 1)

y_cam = z_cam × x_cam = (a,0,−a) × (−a,0,−a)

x: 0·(−a) − (−a)·0 = 0

y: (−a)(−a) − a·(−a) = a² + a² = 1

z: a·0 − 0·(−a) = 0

= (0, 1, 0)

eye가 원점이라 T = 단위행렬, V = R

R = −a 0 −a 0

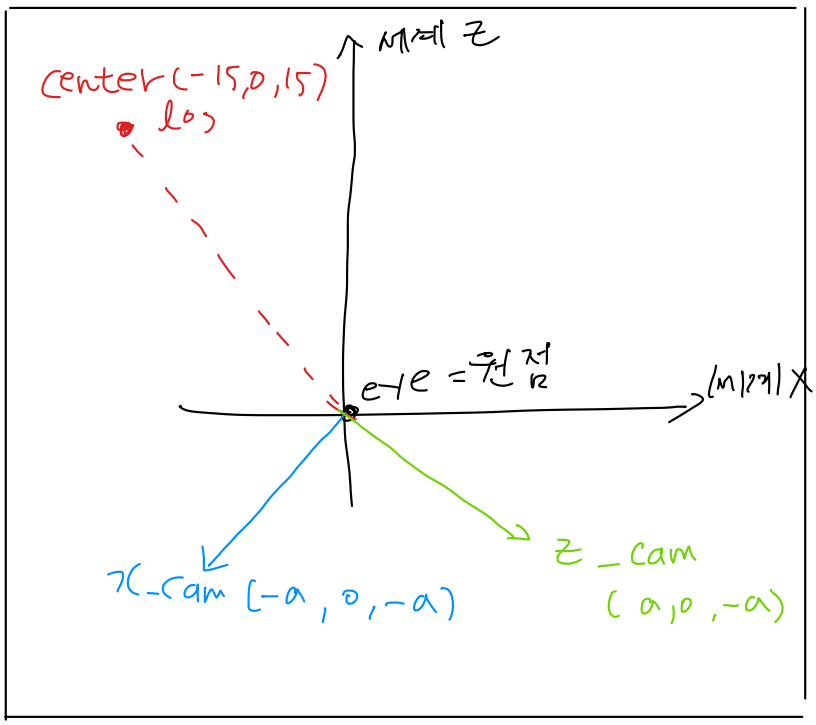
0 1 0 0

a 0 −a 0

0 0 0 1

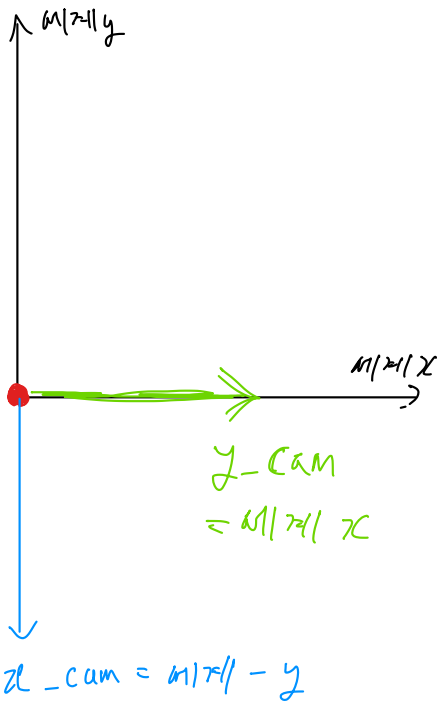
y축 회전 행렬 [cosθ 0 sinθ; 0 1 0; −sinθ 0 cosθ]와 비교하면 cosθ = −1/√2, sinθ = −1/√2 → θ = 225° (또는 −135°).

modeling 변환: `glRotatef(225, 0, 1, 0)` (y축 기준 225° 회전)



(b) gluLookAt(0,0,15, 0,0,0, 1,0,0)

up이 (1,0,0)이라서 (a)와 달리 카메라가 시선축(z)을 중심으로 돌아감



eye = (0,0,15), center = (0,0,0), up = (1,0,0)

los = (0, 0, −15)

z_cam = (0,0,15)/15 = (0, 0, 1)

x_cam = up × z_cam = (1,0,0) × (0,0,1)

x: 0·1 − 0·0 = 0

y: 0·0 − 1·1 = −1

z: 1·0 − 0·0 = 0

= (0, −1, 0)

y_cam = z_cam × x_cam = (0,0,1) × (0,−1,0)

x: 0·0 − 1·(−1) = 1

y: 1·0 − 0·0 = 0

z: 0·(−1) − 0·0 = 0

= (1, 0, 0)

R = 0 −1 0 0

1 0 0 0

0 0 1 0

0 0 0 1

T = translate(0, 0, −15)

V = R × T

카메라 x축이 세계 −y, 카메라 y축이 세계 +x 방향이라 카메라가 z축 둘레로 돌아간 것이다. R을 z축 회전 행렬 [cosθ −sinθ; sinθ cosθ]와 비교하면 cosθ = 0, sinθ = 1 → θ = 90°.

modeling 변환

```
glRotatef(90, 0, 0, 1);
glTranslatef(0, 0, -15);
```

검산: V × eye = R × (0,0,0) = (0,0,0) ✓, V × center = R × (0,0,−15) = (0,0,−15) ✓ (−z축 위, 거리 15), R × up = R × (1,0,0) = (0,1,0) ✓ (up이 화면 위쪽)